FEP-Day2

FEP - Day2

Chatbots

	AI Chatbot	Company	Good for	Official link
1	**ChatGPT**	OpenAI	General AI, coding, writing, analysis	https://chatgpt.com/
2	**Claude**	Anthropic	Writing, coding, document analysis	https://claude.ai/
3	**Gemini**	Google	Multimodal AI, Google ecosystem	https://gemini.google.com/
4	**Microsoft Copilot**	Microsoft	Microsoft 365 and productivity	https://copilot.microsoft.com/
5	**Perplexity**	Perplexity AI	Web research and source-based answers	https://www.perplexity.ai/
6	**Grok**	xAI	General AI and X-integrated experiences	https://grok.com/
7	**DeepSeek**	DeepSeek	Reasoning, coding and general AI	https://chat.deepseek.com/
8	**Meta AI**	Meta	General assistance and Meta ecosystem	https://www.meta.ai/

Creating learning outcomes

Prompt

> You are an experienced faculty member and assessment designer.
>
> Course: [course]
>
> Student level: [level]
>
> Topic: [topic]
>
> Class duration: [duration]
>
> Prior knowledge: [prior knowledge]
>
> Create three measurable learning outcomes for this topic using action verbs. For each outcome, suggest one suitable assessment method.

Check whether the outcomes are measurable. Replace vague verbs such as “understand” with “explain,” “compare,” “design,” or “solve.”

Create the assessment

Prompt

> Using the approved learning outcomes, create a 20-mark assessment.

> Include:
>
> - 4 MCQs worth 1 mark each
> - 2 short-answer questions worth 3 marks each
> - 1 application or case-based question worth 10 marks
>
> For every question, show:
>
> - learning outcome assessed
> - difficulty level
> - marks
> - correct answer or model answer
> - marking guidance
>
> Ensure that at least 40% of the marks test application or analysis.

Quality check:

> Audit this assessment for:
>
> 1. alignment with the learning outcomes
> 2. ambiguous questions
> 3. incorrect answers
> 4. repetition
> 5. balance of difficulty
> 6. total marks
>
> Present the problems in a table, then provide a corrected assessment.

📌 Faculty must verify facts, calculations, code, and the answer key.

Create the rubric

Use the application question or a real assignment from the assessment.

> Create an analytic grading rubric for the 10-mark application question.
>
> Use four criteria and four performance levels: Excellent, Proficient, Developing, Beginning.
>
> Requirements:
>
> - assign marks to every criterion
> - ensure the maximum is exactly 10
> - use observable, measurable descriptors
> - distinguish clearly between adjacent levels
> - avoid vague phrases such as “good understanding”
> - include one short feedback comment for each performance level
>
> Format it as a table.

Rubric review prompt:

> Review this rubric as a strict moderator. Identify overlapping, vague, or non-measurable descriptors. Check that two faculty members could use it consistently. Revise the rubric accordingly.

Generate personalized study material

Use fictional learner profiles—never real student names, IDs, marks, or private information.

> Based on the assessment and its learning outcomes, create study material for these three fictional learners:
>
> A. Foundation learner: has misconceptions and needs simple explanations
>
> B. Practice learner: understands the basics but needs guided application
>
> C. Advanced learner: has mastered the basics and needs challenging extensions
>
> For each learner, provide:
>
> - a short explanation appropriate to their level
> - one worked example
> - three practice questions
> - hints presented separately
> - answers with explanations
> - one recommended next step
>
> Keep the same learning outcomes but vary the scaffolding and difficulty.

For a more authentic adaptive-learning demonstration, introduce diagnostic evidence:

> A fictional student answered Questions 1 and 3 correctly but struggled with Questions 2, 5, and 7.
>
> Infer the likely learning gaps using only this evidence. State where the evidence is insufficient. Create a 30-minute recovery study plan containing:
>
> - concepts to revisit
> - one worked example
> - guided practice
> - an independent check
> - criteria for deciding whether the student is ready to progress

---

Image Creation

Example 1

> Create a clean educational infographic explaining [concept] to [student level] students.
>
> Include:
>
> - A simple definition
> - Three key components
> - One real-life Indian example
> - A visual flow from cause to effect
>
> Use a modern classroom style, high contrast, minimal text, and a 16:9 layout.

Example 2

> Create a 16:9 classroom illustration for first-year engineering students showing a phishing attack.
>
> Show:

> - A fake bank email
> - A student about to click a suspicious link
> - Warning signs highlighted visually
> - A secure verification step
>
> Use a professional flat-vector style, an Indian college setting, and no text inside the image.

Example 3

Draw a rough flowchart on paper or the whiteboard, photograph it, and upload it.

> Transform this rough sketch into a polished educational diagram.
>
> Preserve the original sequence and relationships. Use consistent shapes, clear arrows, accessible colors, and a white background. Leave the labels blank so I can add accurate text later.

Example 4

> Create a realistic educational illustration of an Indian college trying to reduce electricity consumption.
>
> Show classrooms, computer laboratories, solar panels, energy meters, and students analysing usage.
>
> The image should contain several visible clues that students can observe and discuss. Do not include written answers or labels.

Example 5

> Create an educational diagram of [process or system], but deliberately include three plausible conceptual mistakes.
>
> Do not identify the mistakes in the image. Provide the correct answers separately for the faculty member.

Example 6

Create a consistent character or scenario series

> Create a friendly, illustrated character named Professor Asha teaching [subject] in an Indian college classroom.
>
> Professional flat-vector style, blue and orange colour palette, 16:9 format, no text.

Follow with:

> Using the same character and visual style, show Professor Asha conducting a laboratory demonstration.

Example 7

> Create an inclusive image of a diverse Indian engineering team led by a woman engineer. Include varied ages, regional backgrounds, and a colleague with a visible disability. Avoid stereotypes and tokenistic representation.

📌 AI-generated visuals must be reviewed for bias, accuracy, copyright concerns, and misleading realism.

Diagram Creation using Excalidraw (VS Code extension)

Steps

- Open VS Code.
- Install an **Excalidraw** extension from the Extensions view.
- Install or enable your preferred AI coding assistant.
- Open the folder where you want to create diagrams.
- Open the AI assistant's (ChatGPT, Claude, Copilot) chat panel.
- Paste given prompt.

- Approve the assistant's proposed file creation if prompted.
- Wait for it to generate the `.excalidraw` file.
- Find the file in the VS Code Explorer.
- Right-click it and choose **Open With** → **Excalidraw**. Depending on the extension, double-clicking may open it automatically.

Example 1:

Generate a cloud-native application architecture.

> Create an Excalidraw diagram for an e-commerce platform. Include a web app, API gateway, authentication service, product service, order service, PostgreSQL databases, Redis cache, message queue, and payment provider. Group frontend, backend, data, and external services. Add labeled arrows showing request and event flows.

Example 2:

Source code → component diagram. Ask the assistant to inspect a small project and visualize:

- Main modules
- Dependencies
- API endpoints
- Databases and external integrations

> Analyze this repository and create an Excalidraw component diagram showing the major modules and their dependencies. Use different colors for UI, business logic, persistence, and external systems.

Example 3:

User story → sequence diagram

Scenario: Visualize an online purchase flow.

> Create an Excalidraw sequence diagram for "Place an order." Include Customer, Web App, Order API, Inventory Service, Payment Gateway, and Notification Service. Show success and payment-failure paths.

Example 4:

Database requirements → ER diagram

> Create an Excalidraw ER diagram for an online learning platform with User, Course, Enrollment, Lesson, Instructor, Assignment, and Submission entities. Include primary keys, important attributes, cardinalities, and relationship labels.

A useful instruction to append to most prompts is:

> Produce a valid `.excalidraw` JSON file that opens directly in the VS Code Excalidraw extension. Use readable font sizes, non-overlapping elements, consistent spacing, orthogonal connectors where practical, descriptive element labels, and a restrained color palette. Preserve element IDs when modifying an existing diagram.

Diagram Creation using Mermaid (VS Code extension)

> Create a Mermaid flowchart for an online order process: browse products, add to cart, checkout, validate payment, confirm order, and handle payment failure.

> Generate a Mermaid sequence diagram for a user logging in through a web application. Include the browser, frontend, authentication API, database, successful login, and invalid-password response.

> Analyze these classes and create a Mermaid class diagram showing inheritance, composition, attributes, and methods.

> Create a Mermaid ER diagram for an e-commerce database containing Customer, Address, Order, OrderItem, Product, Category, and Payment. Include primary attributes and relationship cardinalities.

> Generate a Mermaid state diagram for an order with the states Draft, Submitted, Paid, Processing, Shipped, Delivered, Cancelled, and Refunded. Include valid transitions and failure paths.

> Create a Mermaid Gantt chart for a six-week application delivery project covering requirements, UI design, development, integration testing, user acceptance testing, and deployment. Show dependencies and overlapping work.

Mind map Creation using Markmap (VS Code extension)

> Create markdown style markmap (https://markmap.js.org/docs/markmap) on Machine Learning. I want detailed mindmap covering all latest topics.